

REPORT · MAY 2026

# Retrieval- Augmented Generation (RAG)

A Visual & Intuitive Guide to Modern AI Retrieval Systems

# Table of Contents

---

|                                                        |    |
|--------------------------------------------------------|----|
| <b>Chapter 1</b> What is RAG? .....                    | 3  |
| <b>Chapter 2</b> The Problem RAG Solves .....          | 5  |
| <b>Chapter 3</b> Embeddings — The Foundation .....     | 7  |
| <b>Chapter 4</b> Chunking Strategies .....             | 9  |
| <b>Chapter 5</b> Vector Databases .....                | 11 |
| <b>Chapter 6</b> Retrieval & Semantic Search .....     | 13 |
| <b>Chapter 7</b> The Complete RAG Pipeline .....       | 15 |
| <b>Chapter 8</b> Building a RAG System in Python ..... | 17 |
| <b>Chapter 9</b> Advanced RAG Patterns .....           | 20 |
| <b>Chapter 10</b> Production Considerations .....      | 23 |
| <b>Appendix</b> References & Further Reading .....     | 25 |

# Chapter 1: What is RAG?

Retrieval-Augmented Generation, or **RAG**, is an architectural pattern that enhances large language models (LLMs) by connecting them to external knowledge sources. Instead of relying solely on what the model learned during training, RAG lets the model *look up* relevant information before generating a response.

Think of it like giving a student an open-book exam. Without RAG, the student must answer from memory alone (closed-book). With RAG, the student can search through reference materials and then compose an answer using both the retrieved facts and their own reasoning ability.

## Why RAG Matters

- LLMs have a fixed knowledge cutoff — they don't know about recent events
- LLMs can hallucinate — they make up plausible-sounding but incorrect information
- LLMs cannot access proprietary or private data by default
- RAG is **cost-effective** — no need to retrain or fine-tune the model
- RAG provides **attribution** — you can cite the source documents used
- RAG is **updatable** — swap in new data without touching the model

**RAG combines the best of two worlds: retrieval (searching for factual information) and generation (composing coherent responses). The retrieval grounds the generation in reality.**

## The Core Idea

A RAG system has three main components that work together in sequence:

1. **Ingest** — Documents are processed, split into chunks, converted to embeddings, and stored in a vector database
2. **Retrieve** — When a user asks a question, it is converted to the same embedding space, and the system searches for the most relevant document chunks
3. **Generate** — The retrieved chunks are injected into the LLM's prompt as context, and the LLM generates a grounded response

## RAG at a Glance

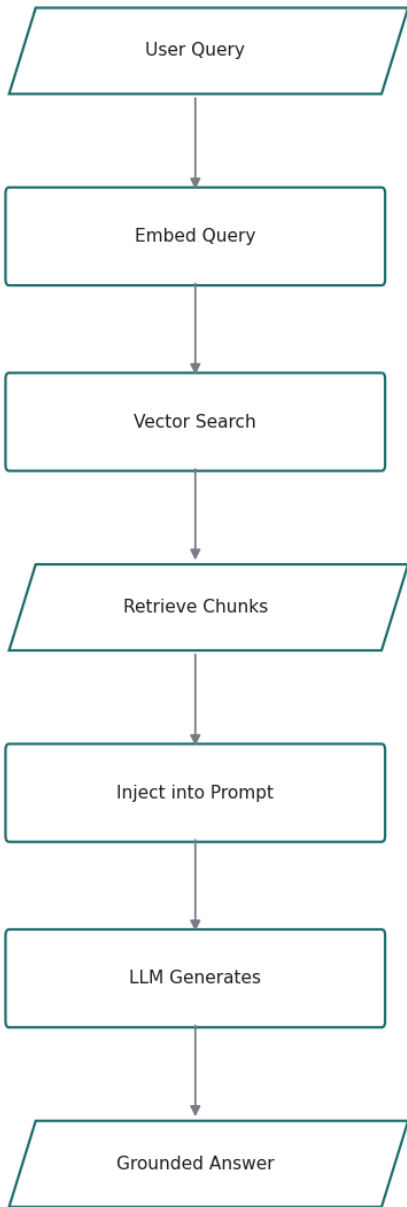


Figure 1: High-level RAG Pipeline — from query to grounded answer

Key Terminology

| Term | Definition | Analogy |
|------|------------|---------|
|------|------------|---------|

|                   |                                                                        |                                                                |
|-------------------|------------------------------------------------------------------------|----------------------------------------------------------------|
| Embedding         | A numerical vector (list of numbers) that captures the meaning of text | A fingerprint for meaning                                      |
| Vector Database   | A database optimized for storing and searching embeddings              | A library organized by topic, not title                        |
| Chunking          | Splitting documents into smaller pieces                                | Cutting a book into paragraphs                                 |
| Semantic Search   | Searching by meaning rather than exact keywords                        | Finding books about 'growing food' vs. searching for 'farming' |
| ANN               | Approximate Nearest Neighbor — fast approximate search                 | Finding a close match without checking every item              |
| Context Injection | Adding retrieved text to the LLM prompt                                | Giving the student reference notes                             |

## Key Takeaways

- RAG is an architecture, not a model — it can be added to any LLM
- RAG grounds LLM responses in factual, retrievable data
- RAG reduces hallucinations, enables citations, and keeps knowledge fresh
- The three pillars are: ingest, retrieve, generate

**Mini Summary: RAG = Search + Read + Write. The system searches for relevant documents, reads them for context, and writes a grounded response.**

# Chapter 2: The Problem RAG Solves

Before understanding why RAG is revolutionary, we must understand the fundamental limitations of large language models. LLMs are remarkable, but they have three critical weaknesses that RAG directly addresses.

## The Knowledge Cutoff Problem

Every LLM is trained on a snapshot of data collected up to a specific date. For example, GPT-4's training data cuts off in April 2023. If you ask it about an event from last week, it literally cannot know the answer — it was never trained on that information.

Think of it like a encyclopedia published in 2020. It's excellent for historical facts, but useless for current events. RAG solves this by giving the model access to up-to-date external data.

## The Hallucination Problem

LLMs are designed to generate plausible-sounding text, not to be factually correct. When they don't know an answer, they often *confabulate* — they make something up that sounds convincing. This is called **hallucination**.

In high-stakes domains like medicine, law, or finance, hallucinations are unacceptable. RAG mitigates this by providing factual context that constrains the model's output.

## The Data Isolation Problem

Organizations have vast amounts of private data — internal documentation, customer records, proprietary research — that no public LLM can access. Fine-tuning on this data is expensive, slow, and requires ML expertise.

RAG provides a practical alternative: keep your data in a vector database, retrieve what's relevant at query time, and never expose the underlying data to the model during training.

## Comparison: Without RAG vs. With RAG

| Scenario                        | Without RAG                                  | With RAG                                        |
|---------------------------------|----------------------------------------------|-------------------------------------------------|
| Ask about latest product launch | I don't have information about that (cutoff) | Retrieves launch docs and summarizes accurately |

|                                  |                                                |                                                    |
|----------------------------------|------------------------------------------------|----------------------------------------------------|
| Ask about internal HR policy     | I can't access internal data (or hallucinates) | Retrieves policy PDF and answers correctly         |
| Ask for a medical recommendation | Generates plausible but possibly wrong answer  | Retrieves latest medical guidelines and cites them |
| Ask with sources                 | No sources to cite                             | Returns answer + source document excerpts          |

## When NOT to Use RAG

- When you need **real-time** responses with zero added latency
- When your queries never require external or up-to-date knowledge
- When you have **highly structured** data better served by traditional databases
- When the retrieval step itself is too costly for your use case

**Engineering Insight: RAG introduces a latency trade-off. Each retrieval adds 100-500ms, but the quality improvement is often dramatic. In practice, the retrieval latency is usually unnoticeable to end users.**

## Key Takeaways

- RAG addresses three fundamental LLM weaknesses: knowledge cutoff, hallucinations, and data isolation
- RAG is the most practical way to give LLMs access to private or real-time data
- RAG adds some latency but dramatically improves factual accuracy

# Chapter 3: Embeddings — The Foundation

Embeddings are the mathematical backbone of every RAG system. To understand RAG, you must first understand what an embedding is and how it captures meaning.

## What is an Embedding?

An **embedding** is a dense vector (a list of numbers) that represents the *semantic meaning* of a piece of text. Unlike traditional keyword-based approaches, embeddings capture meaning, context, and relationships between words and concepts.

Think of it like GPS coordinates for meaning. Just as (40.7128, -74.0060) uniquely identifies New York City, an embedding vector like [0.23, -0.45, 0.89, ...] uniquely identifies the *meaning* of a sentence. Similar sentences have similar coordinates.

## How Embeddings Capture Meaning

Modern embedding models (like **text-embedding-3-small** or **all-MiniLM-L6-v2**) are trained on massive text corpora. During training, the model learns to position semantically similar texts close together in vector space and dissimilar texts far apart.

The key insight: **distance in vector space = difference in meaning**. Two sentences about cats will have vectors that are close together (small distance), while a sentence about cats and one about quantum physics will be far apart (large distance).

## Visualizing Vector Space

While real embeddings have 384 to 3072 dimensions, we can visualize the concept in 2D. Clusters of related concepts form naturally:

*In this simplified 2D projection of an embedding space, notice how "dog", "puppy", and "canine" cluster together near the bottom-left, while "car", "truck", and "vehicle" form their own cluster upper-right. The distance between clusters reflects semantic difference.*

## Common Embedding Models

| Model                  | Dimensions | Context Length | Use Case                        |
|------------------------|------------|----------------|---------------------------------|
| text-embedding-3-small | 1536       | 8191 tokens    | General purpose, cost-effective |



|                                 |      |             |                            |
|---------------------------------|------|-------------|----------------------------|
| text-embedding-3-large          | 3072 | 8191 tokens | High precision, enterprise |
| all-MiniLM-L6-v2                | 384  | 256 tokens  | Fast, local, open-source   |
| BAAI/bge-base-en-v1.5           | 768  | 512 tokens  | Balanced performance       |
| intfloat/e5-mistral-7b-instruct | 4096 | 4096 tokens | State-of-the-art accuracy  |

**Important: The input text and the query must use the same embedding model to produce compatible vectors. Always use the same embedding model for indexing and retrieval.**

Key Takeaways

- Embeddings are numerical vectors that capture semantic meaning
- Similar meanings = similar vectors (close together in vector space)
- Embedding dimension ranges from 384 to 3072 depending on the model
- The same embedding model must be used for both indexing and retrieval
- Open-source models (MiniLM, BGE, E5) make local RAG systems practical

# Chapter 4: Chunking Strategies

Before documents can be searched, they must be split into manageable pieces called **chunks**. Chunking seems simple — just split text — but the strategy you choose dramatically impacts retrieval quality.

## Why Chunking Matters

Think of chunking as deciding the *granularity* of your knowledge base. Too large, and each chunk contains multiple topics — the embedding becomes muddled. Too small, and individual chunks lack sufficient context to be useful.

The goal is to create chunks that are **semantically self-contained** — each chunk should express one complete idea or piece of information.

## Chunking Approaches Compared

| Strategy            | How It Works                                          | Best For               | Downside                  |
|---------------------|-------------------------------------------------------|------------------------|---------------------------|
| Fixed Size          | Split every N characters/tokens with optional overlap | Simple documents, logs | May cut sentences in half |
| Recursive Character | Split on natural boundaries (\n\n → \n → . → ! → ?)   | General text, articles | No semantic awareness     |
| Semantic Chunking   | Use embeddings to detect topic boundaries             | Complex documents      | Slower, more expensive    |
| Document-Based      | Use document structure (sections, paragraphs)         | PDFs, Markdown, HTML   | Requires structured input |
| Agentic Chunking    | LLM decides where to split                            | High-value documents   | Slowest, most expensive   |

## The Overlap Trick

A simple but effective technique: when chunking, overlap consecutive chunks by 10-20%. This ensures that information near chunk boundaries isn't lost:

Chunk 1: [The cat sat on the mat. The mat was]

*Chunk 2: [The mat was old and worn. The cat purred.]*

The overlapping phrase "The mat was" ensures both chunks carry context across the boundary.

## Chunk Size Trade-offs

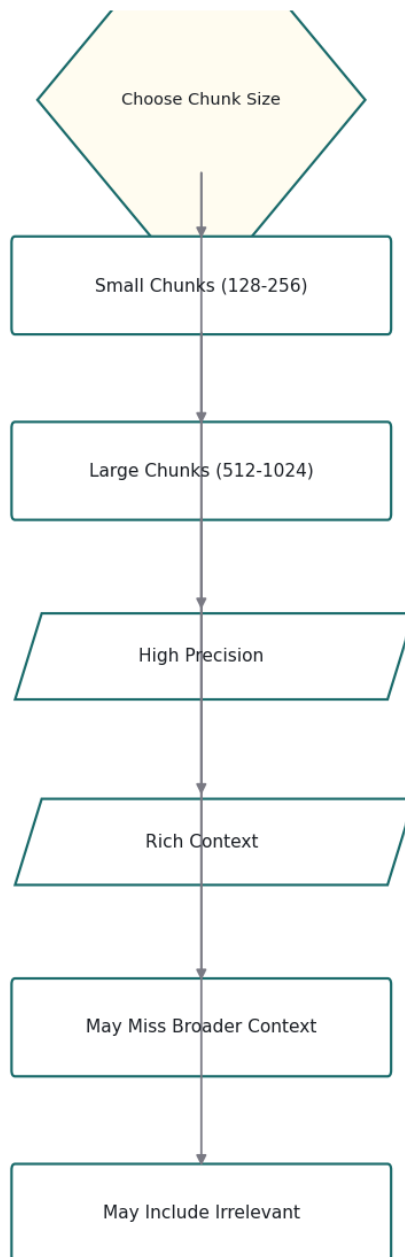


Figure 2: The trade-off between chunk size, precision, and context richness

## Practical Guidelines

- **Start with 512 tokens** — this is a well-balanced default for most use cases
- Use **10-20% overlap** to avoid information loss at boundaries
- Chunk at **semantic boundaries** (paragraphs, sections) not arbitrary token counts
- For Q&A;, aim for chunks that each contain **one question-answerable fact**
- **Test different chunk sizes** — there is no universally correct value

**Production Tip:** Use LangChain's `RecursiveCharacterTextSplitter` as your starting point. It splits on natural delimiters (double newline → single newline → sentence end) and automatically handles overlap.

## Key Takeaways

- Chunking determines the granularity of your knowledge base
- 512 tokens with 20% overlap is a strong default
- Chunk at natural semantic boundaries, not arbitrary positions
- Overlap prevents information loss at chunk boundaries
- Different document types may need different chunking strategies

# Chapter 5: Vector Databases

Once you have converted documents into embedding vectors, you need a place to store them — and more importantly, a way to search them by similarity. This is what **vector databases** do.

## What Makes Vector Databases Special?

Traditional databases (PostgreSQL, MySQL) are optimized for **exact matches** — they find rows where a column equals a specific value. Vector databases are optimized for **similarity search** — they find vectors closest to a given query vector.

Think of it like the difference between a library catalog (traditional DB) and a friend who knows your taste (vector DB). The catalog tells you exactly where a specific book is. Your friend says, "Based on what you liked before, you'll probably enjoy these."

## Popular Vector Databases Compared

| Database | Type                 | Search Algorithm              | Best For                           |
|----------|----------------------|-------------------------------|------------------------------------|
| ChromaDB | Embedded (Python)    | Flat + HNSW                   | Prototyping, small-medium projects |
| FAISS    | Library (not a DB)   | Multiple (IVF, HNSW)          | High-performance, research         |
| Pinecone | Managed Cloud        | HNSW (proprietary)            | Enterprise, production             |
| Weaviate | Standalone Server    | HNSW + custom                 | Full-featured, hybrid search       |
| Qdrant   | Standalone Server    | HNSW                          | Rust-based, high throughput        |
| Milvus   | Distributed          | Multiple (IVF, HNSW, DiskANN) | Large-scale, sharded               |
| pgvector | PostgreSQL Extension | IVFFlat, HNSW                 | PostgreSQL users, hybrid           |

## How Vector Search Works

Behind the scenes, vector databases use specialized index structures to enable fast similarity search. The most common approach is **HNSW** (Hierarchical Navigable Small World):

- 1. Vectors are stored in a graph structure organized by layers
- 2. The top layer has few nodes (long-range connections)
- 3. Lower layers have more nodes (fine-grained connections)
- 4. Search starts at the top layer and descends — like taking a highway then local roads
- 5. This achieves  $O(\log n)$  search complexity — millions of vectors searched in milliseconds

Distance Metrics

| Metric            | Formula                                 | When to Use                       |
|-------------------|-----------------------------------------|-----------------------------------|
| Cosine Similarity | $\cos(A,B) = A \cdot B /  A  \cdot  B $ | Semantic similarity (most common) |
| Euclidean (L2)    | $d = \sqrt{\sum (A_i - B_i)^2}$         | Dense embeddings, spatial data    |
| Dot Product       | $d = \sum A_i \cdot B_i$                | When vectors are normalized       |

**Important: Most modern embedding models produce normalized vectors. For normalized vectors, cosine similarity and dot product produce the same ranking. Check your embedding model's documentation.**

Key Takeaways

- Vector databases enable similarity search over embeddings
- HNSW is the most widely used indexing algorithm for vector search
- ChromaDB and FAISS are excellent choices for getting started locally
- Distance metrics matter — cosine similarity is the default for semantic search
- For production, consider managed services (Pinecone) or full-featured engines (Weaviate, Qdrant)

# Chapter 6: Retrieval & Semantic Search

Retrieval is the heart of any RAG system. It determines *which* pieces of information the LLM will see. A good retrieval system finds the right context; a bad one feeds irrelevant noise.

## Semantic Search vs. Keyword Search

Traditional **keyword search** (like grep or SQL LIKE) looks for exact word matches. If you search for "canine", you won't find documents mentioning "dog". **Semantic search** understands meaning — it finds "dog" documents even when you search for "canine".

Think of keyword search as a jigsaw puzzle — you need the exact piece. Semantic search is like a magnet — it pulls in all nearby pieces regardless of their shape.

## ANN Search: Speed vs. Accuracy

**Approximate Nearest Neighbor (ANN)** search is the technique behind all modern vector databases. Instead of measuring distance to every single vector (which would be  $O(n)$  and prohibitively slow for millions of vectors), ANN uses smart indexing to find approximate neighbors in  $O(\log n)$  time.

The trade-off: ANN is *approximate*, not exact. You might miss the true nearest neighbor, but you'll find one that's close enough — and you'll do it 100-1000x faster.

## Retrieval Quality Metrics

| Metric      | What It Measures                                               | Target |
|-------------|----------------------------------------------------------------|--------|
| Precision@K | Of top-K results, how many are relevant?                       | > 0.8  |
| Recall@K    | Of all relevant documents, how many are in top-K?              | > 0.7  |
| MRR         | Mean Reciprocal Rank — how early is the first relevant result? | > 0.9  |
| NDCG@K      | Normalized Discounted Cumulative Gain — ranking quality        | > 0.8  |

## Hybrid Search: Best of Both Worlds

**Hybrid search** combines semantic search (embedding similarity) with keyword search (BM25) and fuses the results. This catches cases where exact keyword matches matter (like product codes or names) while still benefiting from semantic understanding.

The fusion is typically done with **Reciprocal Rank Fusion (RRF)**, which combines rankings from multiple sources:

RRF score =  $1/(60 + \text{rank\_semantic}) + 1/(60 + \text{rank\_keyword})$

## The Retrieval Flow



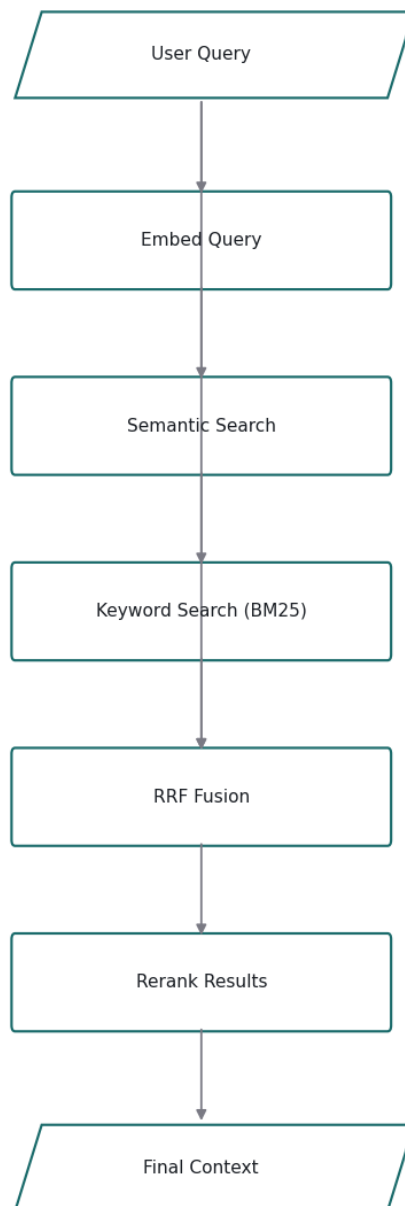


Figure 3: Hybrid retrieval pipeline combining semantic and keyword search with RRF fusion

**Production Tip: A reranker step (also called a cross-encoder) re-scores the top 20-50 results with a more accurate model. This dramatically improves precision at the cost of a small latency increase. Use models like BAAI/bge-reranker-v2-m3.**

## Key Takeaways

- Semantic search finds by meaning, keyword search finds by exact match
- ANN makes vector search practical at scale ( $O(\log n)$  vs.  $O(n)$ )
- Hybrid search (semantic + BM25 + RRF) is more robust than either alone
- A reranker step before the LLM significantly improves final quality
- Measure retrieval quality with Precision@K and Recall@K

# Chapter 7: The Complete RAG Pipeline

Now that we understand each component, let's assemble the complete RAG pipeline — from raw documents to grounded answers. This chapter shows the full system architecture and data flow.

## The Two Phases

Every RAG system operates in two distinct phases:

- **Indexing Phase (offline):** Process and store documents for future retrieval
- **Query Phase (online/real-time):** Retrieve relevant context and generate answers

### Phase 1: Indexing (Offline)

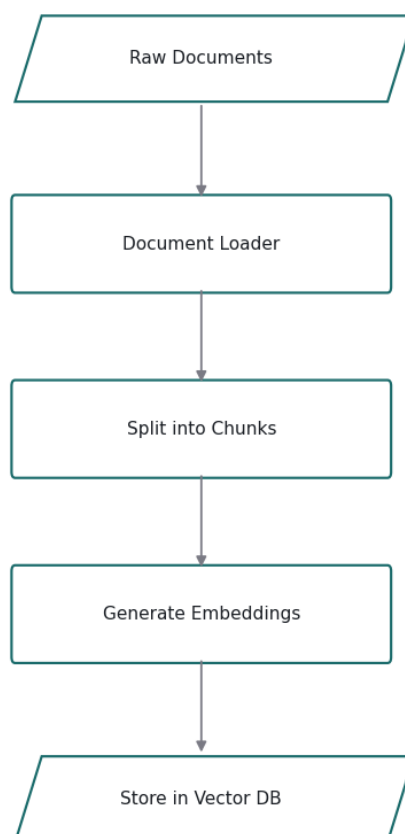


Figure 4: Indexing Phase — preparing documents for search

Phase 2: Query (Online)

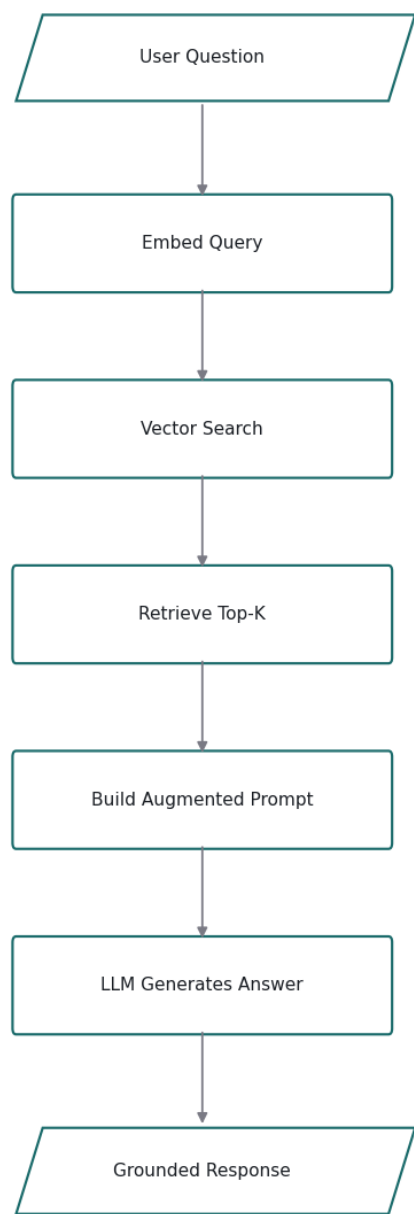


Figure 5: Query Phase — retrieving context and generating answers

## End-to-End Architecture

Here is the complete system architecture showing how a production RAG system fits together:

## The Augmented Prompt

The key innovation in RAG is how retrieved context is injected into the prompt. A typical augmented prompt looks like this:

TEXT

```
SYSTEM: You are a helpful assistant. Use the following context
to answer the user's question. If you cannot find the answer
in the context, say so – do not make up information.

CONTEXT:
---
[Document 1]: The Python garbage collector uses reference
counting and a cycle detector.
---
[Document 2]: Memory management in Python is automatic.
Objects are deallocated when their reference count reaches zero.
---
[Document 3]: The gc module provides control over the
cyclic garbage collector.
---

USER: How does Python handle memory management?

ASSISTANT:
```

**Important: Always instruct the LLM to refuse answering if the context doesn't contain relevant information. This prevents the model from falling back on its training data and potentially hallucinating.**

## Key Takeaways

- RAG has two phases: Indexing (offline) and Query (online)
- The indexing pipeline: Load → Split → Embed → Store
- The query pipeline: Embed query → Search → Retrieve → Augment prompt → Generate
- The augmented prompt contains retrieved context + the original question
- Always instruct the LLM to only answer from the provided context

# Chapter 8: Building a RAG System in Python

In this chapter, we'll build a complete RAG system step by step using Python. We'll use **LangChain** for orchestration, **ChromaDB** for the vector store, and open-source embedding models.

## Setup & Installation

PYTHON

```
pip install langchain langchain-community langchain-chroma
pip install chromadb sentence-transformers pypdf
```

## Step 1: Load a PDF Document

PYTHON

```
from langchain_community.document_loaders import PyPDFLoader

loader = PyPDFLoader("handbook.pdf")
documents = loader.load()
print(f"Loaded {len(documents)} pages")
```

## Step 2: Split into Chunks

PYTHON

```
from langchain.text_splitter import RecursiveCharacterTextSplitter

splitter = RecursiveCharacterTextSplitter(
    chunk_size=512,
    chunk_overlap=64,
    separators=["\n\n", "\n", ".", "!", "?"],
    length_function=len,
)

chunks = splitter.split_documents(documents)
print(f"Created {len(chunks)} chunks")
```

## Step 3: Create Embeddings & Vector Store

PYTHON

```
from langchain_chroma import Chroma
from langchain_community.embeddings import HuggingFaceEmbeddings

embeddings = HuggingFaceEmbeddings(
    model_name="all-MiniLM-L6-v2",
    model_kwargs={"device": "cpu"},
    encode_kwargs={"normalize_embeddings": True},
)

vectorstore = Chroma.from_documents(
    documents=chunks,
    embedding=embeddings,
    persist_directory="./chroma_db",
)

print(f"Stored {vectorstore._collection.count()} vectors")
```

## Step 4: Set Up the Retriever

PYTHON

```
retriever = vectorstore.as_retriever(
    search_type="similarity",
    search_kwargs={"k": 4},
)
```

## Step 5: Create the RAG Chain

PYTHON

```
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.runnables import RunnablePassthrough
from langchain_openai import ChatOpenAI

template = """You are a helpful assistant. Answer using only
the provided context. If the context lacks the information,
say you don't know.

Context:
{context}

Question: {question}
Answer: """

prompt = ChatPromptTemplate.from_template(template)
llm = ChatOpenAI(model="gpt-4o-mini", temperature=0)

def format_docs(docs):
    return "\n\n".join(d.page_content for d in docs)

rag_chain = (
    {"context": retriever | format_docs,
     "question": RunnablePassthrough()}
    | prompt
    | llm
)
```

## Step 6: Ask Questions!

PYTHON

```
result = rag_chain.invoke(
    "What is the company policy on remote work?"
)
print(result.content)
```

## Full Working Example with FAISS

PYTHON



```
from langchain_community.vectorstores import FAISS
from langchain_community.document_loaders import TextLoader
from langchain.text_splitter import CharacterTextSplitter
from langchain_community.embeddings import HuggingFaceEmbeddings
from langchain_openai import ChatOpenAI
from langchain.chains import RetrievalQA

loader = TextLoader("knowledge_base.txt")
docs = loader.load()

text_splitter = CharacterTextSplitter(
    chunk_size=500, chunk_overlap=50
)
chunks = text_splitter.split_documents(docs)

embeddings = HuggingFaceEmbeddings(
    model_name="all-MiniLM-L6-v2"
)

vectorstore = FAISS.from_documents(chunks, embeddings)

qa = RetrievalQA.from_chain_type(
    llm=ChatOpenAI(model="gpt-4o-mini", temperature=0),
    chain_type="stuff",
    retriever=vectorstore.as_retriever(search_kwargs={"k": 3}),
    return_source_documents=True,
)

response = qa.invoke(
    {"query": "What are the main features?"}
)
print(response["result"])
print(f"Sources: {len(response['source_documents'])}")
```

**Engineering Insight:** The `return_source_documents=True` flag lets you inspect which documents the model used to generate its answer. This is crucial for debugging and building trust with users through citations.

## Key Takeaways

- LangChain provides clean abstractions for building RAG pipelines
- ChromaDB is ideal for prototyping; FAISS offers more speed for local use
- Always include a "don't know" instruction in your prompt to prevent hallucination
- Return source documents for debugging and attribution
- Open-source embedding models (all-MiniLM-L6-v2) work well for local development

## Chapter 9: Advanced RAG Patterns

---

Once you master basic RAG, there are several advanced patterns that address specific limitations. Three of the most important are **Agentic RAG**, **Graph RAG**, and **HyDE**.

### Agentic RAG: Let the LLM Drive

In basic RAG, the retriever always runs, always retrieves the same number of documents (k), and always injects them into the prompt. **Agentic RAG** gives the LLM agency over the retrieval process:

1. The LLM decides **whether** to retrieve — it can skip retrieval for simple queries
2. The LLM decides **what** to search for — it can reformulate or decompose queries
3. The LLM decides **when** to retrieve — it can do multiple rounds of research
4. The LLM can use **tools** — calling APIs, querying databases, or browsing the web

Think of it like a research assistant who doesn't just hand you pre-selected documents but actively decides what to look up, in what order, and whether more information is needed.

### Agentic RAG Flow

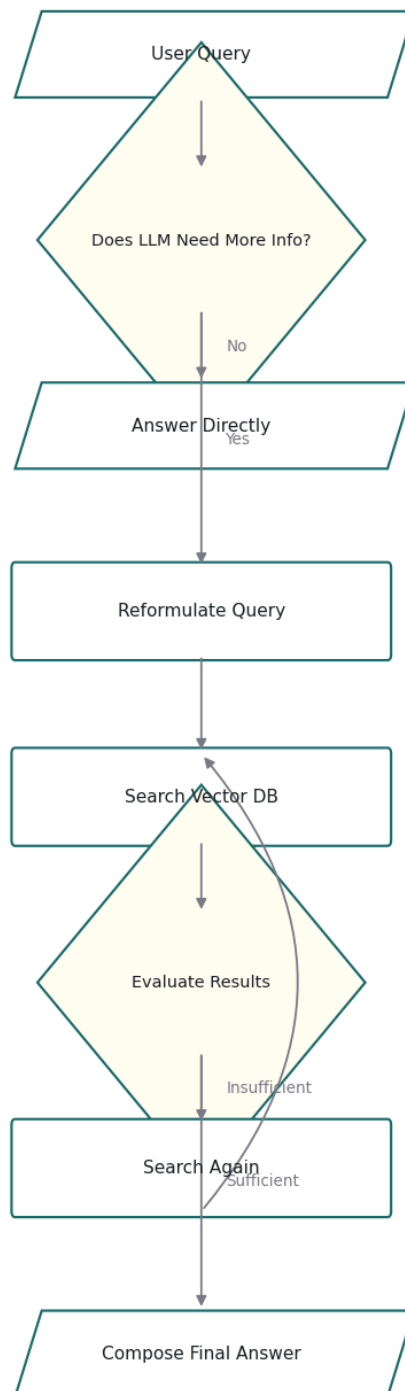


Figure 6: Agentic RAG — the LLM controls its own retrieval process

### Graph RAG: Structured Knowledge

**Graph RAG** (developed by Microsoft) extracts entities and relationships from documents to build a knowledge graph. Instead of searching over raw text chunks, Graph RAG queries the graph structure, enabling:

- **Multi-hop reasoning:** "What products does the CEO's former company make?"
- **Global summarization:** Summarize themes across the entire knowledge base
- **Relationship-aware retrieval:** Find documents connected through entities

Think of it as the difference between a file cabinet (basic RAG) and a mind map (Graph RAG). The file cabinet stores documents; the mind map shows how everything connects.

### HyDE: Hypothetical Document Embeddings

**HyDE** (Hypothetical Document Embeddings) is a clever technique: instead of searching with the user's question directly, you first ask an LLM to generate a *hypothetical answer*, then use that answer as the search query.

The insight: queries and documents live in different regions of embedding space. A question like "What causes inflation?" is structurally different from a document about inflation. By generating a hypothetical document first, HyDE bridges this gap.

### Patterns Comparison

| Pattern       | Best For                                        | Complexity | Trade-off                         |
|---------------|-------------------------------------------------|------------|-----------------------------------|
| Basic RAG     | Simple Q&A;,<br>straightforward<br>retrieval    | Low        | Fixed retrieval, no<br>reasoning  |
| Agentic RAG   | Complex, multi-step<br>research tasks           | High       | Slower, more LLM<br>calls         |
| Graph RAG     | Connected data,<br>entity relationships         | High       | Requires graph<br>construction    |
| HyDE          | Improving retrieval<br>for ambiguous<br>queries | Medium     | Extra LLM call per<br>query       |
| Hybrid Search | Mixed content<br>(keywords +<br>semantics)      | Medium     | Two search systems<br>to maintain |

**Production Tip: Start with basic RAG and hybrid search. Only add Agentic or Graph RAG if your evaluation shows clear gaps that simpler approaches cannot fill. Complexity should be earned, not assumed.**

## Key Takeaways

- Agentic RAG gives the LLM control over when and what to retrieve
- Graph RAG extracts entities and relationships for structured reasoning
- HyDE improves retrieval by searching with a hypothetical answer instead of the question
- Start simple — add complexity only when evaluation proves it necessary
- The RAG ecosystem is rapidly evolving; these patterns represent the current frontier

# Chapter 10: Production Considerations

Building a RAG system that works on your laptop is one thing. Running it reliably in production is another. This chapter covers the critical considerations for production-ready RAG.

## Performance & Latency

A production RAG system must balance **speed**, **quality**, and **cost**. Here are the typical latency contributions:

| Component                | Typical Latency | Optimization                      |
|--------------------------|-----------------|-----------------------------------|
| Query embedding          | 10-50ms         | Use smaller models, GPU inference |
| Vector search (ANN)      | 5-50ms          | Tune HNSW ef_search parameter     |
| Reranker (cross-encoder) | 50-200ms        | Only rerank top-20 results        |
| LLM generation           | 500ms-5s        | Use smaller models, streaming     |
| Total                    | ~1-5s           | —                                 |

## Evaluation: Is Your RAG System Good?

You cannot improve what you don't measure. A comprehensive RAG evaluation framework should track:

- **Retrieval Quality:** Precision@K, Recall@K, MRR
- **Generation Quality:** Faithfulness (does output match context?), Answer Relevance
- **End-to-End Quality:** Human evaluation, task completion rate
- **Operational Metrics:** p50/p95/p99 latency, error rate, cost per query

Tools like **RAGAS** (RAG Assessment) provide automated evaluation metrics. TruLens and LangSmith offer observability and tracing.

## Security Considerations

- **Data exfiltration:** Ensure retrieved documents cannot be leaked through prompt injection

- **Access control:** Users should only retrieve documents they have permission to see
- **Content filtering:** Screen retrieved content for PII, toxic content, or policy violations
- **Prompt injection:** Protect against poisoned documents that manipulate the LLM
- **Rate limiting:** Protect your vector DB and LLM API from abuse

**Production Tip: Implement a "human in the loop" for high-stakes retrieval. Flag ambiguous or low-confidence retrievals for human review. This is especially critical in healthcare, legal, and financial applications.**

RAG Maturity Model

| Level | Name       | Characteristics                                  |
|-------|------------|--------------------------------------------------|
| L1    | Basic      | Single chunk retrieval, no reranking, fixed k    |
| L2    | Structured | Hybrid search, chunk overlap, metadata filtering |
| L3    | Optimized  | Reranker, query expansion, dynamic chunk sizes   |
| L4    | Adaptive   | Agentic retrieval, self-correction, multi-hop    |
| L5    | Autonomous | Graph RAG, tool use, continuous learning         |

RAG Pipeline: The Full Picture

A production RAG system consists of two parallel pipelines — indexing (offline) and query (online). Here is the complete architecture with all components:

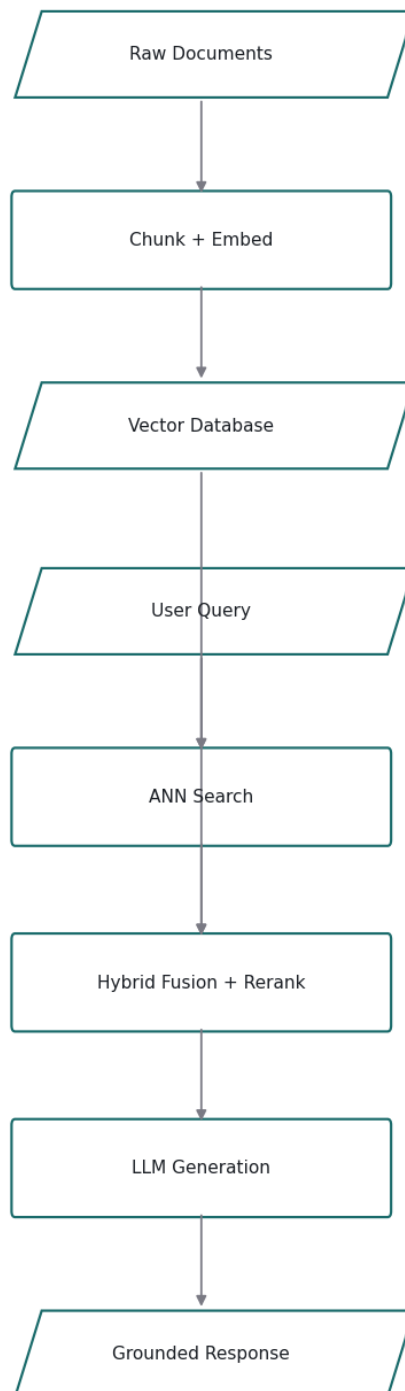


Figure 7: Complete production RAG architecture showing both indexing and query paths



The indexing pipeline (top) processes raw documents into searchable vectors. The query pipeline (bottom) retrieves the most relevant context and feeds it to the LLM for grounded generation.

## Key Takeaways

- Production RAG requires monitoring latency, quality, and cost
- Evaluate both retrieval quality and generation faithfulness
- Security is critical — access control, content filtering, and prompt injection protection
- Use the RAG Maturity Model to plan your evolution
- Start simple, measure everything, add complexity only when data justifies it

# Appendix: References & Further Reading

---

This appendix provides references to the key papers, tools, and resources that shaped modern RAG systems. Use these as a starting point for deeper study.

## Foundational Papers

- [1] Lewis, P. et al. (2020). "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks." NeurIPS. — *The original RAG paper introducing the concept.*
- [2] Karpukhin, V. et al. (2020). "Dense Passage Retrieval for Open-Domain Question Answering." EMNLP. — *Foundation for dense embedding-based retrieval.*
- [3] Gao, L. et al. (2022). "Precise Zero-Shot Dense Retrieval without Relevance Labels." — *The HyDE paper.*
- [4] Edge, D. et al. (2024). "From Local to Global: A Graph RAG Approach to Query-Focused Summarization." Microsoft Research.
- [5] Malkov, Y. & Yashunin, D. (2016). "Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs." IEEE TPAMI. — *The HNSW paper.*

## Tools & Frameworks

- **LangChain** — The most popular framework for building RAG applications ([python.langchain.com](https://python.langchain.com))
- **LlamaIndex** — A data framework specifically designed for RAG ([llamaindex.ai](https://llamaindex.ai))
- **ChromaDB** — Embedded vector database for prototyping ([trychroma.com](https://trychroma.com))
- **FAISS** — High-performance vector search library from Meta ([faiss.ai](https://faiss.ai))
- **RAGAS** — Automated evaluation framework for RAG pipelines ([ragas.io](https://ragas.io))
- **Weaviate / Qdrant / Pinecone** — Production-grade vector database services
- **sentence-transformers** — Open-source embedding models ([sbert.net](https://sbert.net))

## Learning Resources

- **OpenAI Cookbook** — Practical RAG examples and patterns
- **LangChain RAG Tutorials** — Official documentation with code examples
- **Andrej Karpathy's "Intro to Large Language Models"** — Foundational LLM knowledge
- **HuggingFace NLP Course** — Free course covering embeddings and transformers

**Final Thought: RAG is not a solved problem — it is an active research area. The field is evolving weekly. Follow the papers, experiment with the tools, and always measure your results. The best RAG system is the one that solves your specific problem.**

This document was generated as an educational guide. For questions, corrections, or contributions, refer to the RAG community resources listed above.